

05 Inverse of a Matrix



5.3 Solving the Inverse by Hand

Using the adjoint-matrix method to hand-compute the inverse of a 3×3 matrix, step by step

Linear Algebra Made Easy and Visual | Open-source code & drafts: <https://github.com/Visualize-ML>

Core Skills You Will Build

1

A square matrix is invertible only when its determinant is nonzero

2

Delete the matching row and column to build each element's minor matrix

3

Take the determinant of each minor matrix to get the minor

4

Adjust each minor's sign based on its position to get the cofactor

5

The cofactors together form the cofactor matrix

6

Transposing the cofactor matrix gives the adjoint (adjugate) matrix

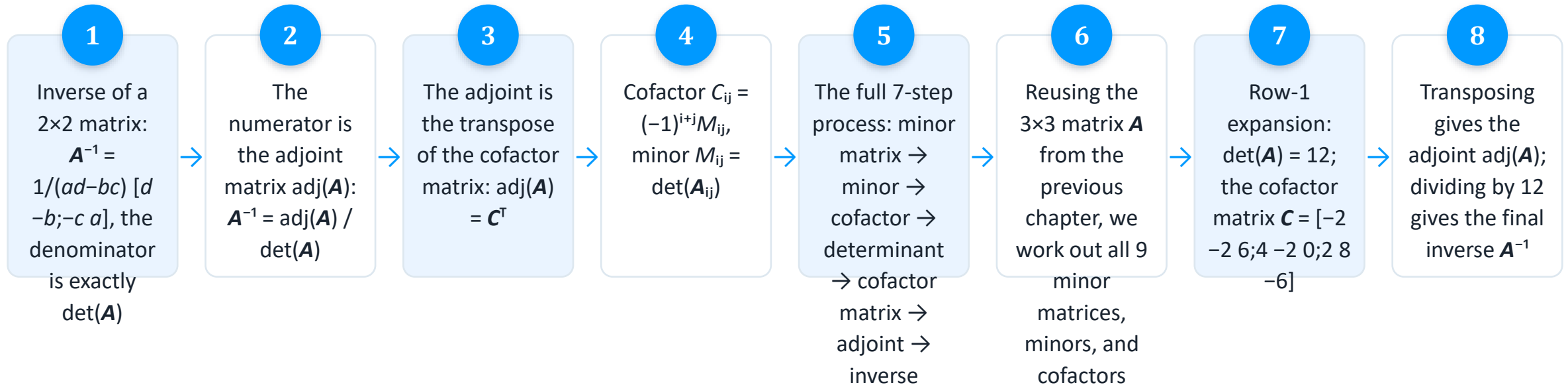
7

The inverse is the adjoint matrix divided by the determinant

8

Review: computing a determinant via Laplace expansion

How This Section Unfolds



Starting with the 2×2 Inverse Formula

$$\mathbf{A} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \Rightarrow \mathbf{A}^{-1} = 1/(ad-bc) \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$$

- As covered earlier, a nonzero determinant is necessary and sufficient for $\mathbf{A}$ to be invertible — the denominator $ad-bc$ above is exactly $\mathbf{A}$'s determinant $\det(\mathbf{A})$
- If the denominator is the determinant, what's the numerator? That numerator is called $\mathbf{A}$'s adjoint matrix, written $\text{adj}(\mathbf{A})$

INVERSE = ADJOINT ÷ DETERMINANT

$$\mathbf{A}^{-1} = \text{adj}(\mathbf{A}) / \det(\mathbf{A})$$

$$\mathbf{A}^{-1} = \text{adj}(\mathbf{A}) / \det(\mathbf{A})$$

- If $\mathbf{A}$ is invertible, its inverse can be written as the adjoint matrix divided by the determinant — so to find the inverse, we first find the determinant, then the adjoint
- The adjoint is defined as $\text{adj}(\mathbf{A}) = \mathbf{C}^T$, where $\mathbf{C}$ is the cofactor matrix; the adjoint is simply the cofactor matrix's transpose

The Cofactor Matrix Is Built from Every Element's Cofactor

$$C_{ij} = (-1)^{i+j} M_{ij}, M_{ij} = \det(\mathbf{A}_{ij})$$

- Each cofactor comes from deleting that element's row and column, taking the determinant of the resulting minor matrix $\mathbf{A}_{ij}$ (the minor M_{ij}), then adjusting the sign based on position
- The minor matrix $\mathbf{A}_{ij}$ is what's left of $\mathbf{A}$ after removing row i and column j ; the minor M_{ij} is simply that minor matrix's determinant

Working Backward Through the Whole Procedure

a) Expanding the determinant only uses one row or column — extract that row/column's minor matrices (square)

b) From each minor matrix, compute every element's minor (a scalar)

c) Compute every element's cofactor (a scalar) — the sign-adjusted minor

d) Use Laplace expansion to compute the determinant

e) Using the results from c), assemble the cofactor matrix

f) Transpose the cofactor matrix to get the adjoint matrix

g) Finally, multiply the adjoint by the reciprocal of the determinant to get the inverse

Note The determinant only needs minor matrices along one row/column; the cofactor matrix needs every element's minor matrix

Given a 3×3 Matrix A, Compute Its Inverse Step by Step

$$A = [1 \ 2 \ 3; 3 \ 0 \ 1; 1 \ 2 \ 1]$$

- This example reuses the 3×3 matrix from the previous section's hand-computed determinant — most of the work below already appeared there, which makes this a great review of minors, cofactors, and Laplace expansion
- Next, we extract all 9 elements' minor matrices, then compute their minors and cofactors one by one

Minor Matrices, Minors, and Cofactors for All 9 Elements

Element	Minor Matrix $A_{i,j}$	Minor $M_{i,j}$	Cofactor $C_{i,j}$
$a_{1,1}$	$\begin{bmatrix} 0 & 1; 2 & 1 \end{bmatrix}$	-2	-2
$a_{1,2}$	$\begin{bmatrix} 3 & 1; 1 & 1 \end{bmatrix}$	2	-2
$a_{1,3}$	$\begin{bmatrix} 3 & 0; 1 & 2 \end{bmatrix}$	6	6
$a_{2,1}$	$\begin{bmatrix} 2 & 3; 2 & 1 \end{bmatrix}$	-4	4
$a_{2,2}$	$\begin{bmatrix} 1 & 3; 1 & 1 \end{bmatrix}$	-2	-2
$a_{2,3}$	$\begin{bmatrix} 1 & 2; 1 & 2 \end{bmatrix}$	0	0
$a_{3,1}$	$\begin{bmatrix} 2 & 3; 0 & 1 \end{bmatrix}$	2	2
$a_{3,2}$	$\begin{bmatrix} 1 & 3; 3 & 1 \end{bmatrix}$	-8	8
$a_{3,3}$	$\begin{bmatrix} 1 & 2; 3 & 0 \end{bmatrix}$	-6	-6

Expanding Along Row 1

$$\det(\mathbf{A}) = a_{1,1}C_{1,1} + a_{1,2}C_{1,2} + a_{1,3}C_{1,3} = 1 \times (-2) + 2 \times (-2) + 3 \times 6 = 12$$

- Plugging in row 1's cofactors from the table — $C_{1,1} = -2$, $C_{1,2} = -2$, $C_{1,3} = 6$ — gives $\det(\mathbf{A}) = 12$
- $\det(\mathbf{A}) \neq 0$, so $\mathbf{A}$ is invertible — next we assemble the cofactor matrix and take its transpose to get the adjoint

Arranging All 9 Cofactors into Matrix $\mathbf{C}$

$$\mathbf{C} = [C_{1,1} \ C_{1,2} \ C_{1,3}; C_{2,1} \ C_{2,2} \ C_{2,3}; C_{3,1} \ C_{3,2} \ C_{3,3}] = \begin{bmatrix} -2 & -2 & 6 \\ 4 & -2 & 0 \\ 2 & 8 & -6 \end{bmatrix}$$

- Placing the 9 cofactors from the table back into their original row/column positions gives the cofactor matrix $\mathbf{C}$
- Next, simply transposing $\mathbf{C}$ gives the adjoint matrix $\text{adj}(\mathbf{A})$

The Adjoint Is the Cofactor Matrix's Transpose

$$\text{adj}(\mathbf{A}) = \mathbf{C}^T = [-2 \ 4 \ 2; -2 \ -2 \ 8; 6 \ 0 \ -6]$$

- By definition $\text{adj}(\mathbf{A}) = \mathbf{C}^T$ — swapping $\mathbf{C}$'s rows and columns gives the adjoint matrix
- Notice that $\mathbf{C}$'s first row $[-2 \ -2 \ 6]$ becomes $\text{adj}(\mathbf{A})$'s first column $[-2; -2; 6]$ — that's exactly what transposing does

COMPUTING THE INVERSE

$$\mathbf{A}^{-1} = \text{adj}(\mathbf{A}) / \det(\mathbf{A})$$

$$\mathbf{A}^{-1} = (1/12) \times [-2 \ 4 \ 2; -2 \ -2 \ 8; 6 \ 0 \ -6]$$

- Plugging in $\det(\mathbf{A}) = 12$ and $\text{adj}(\mathbf{A})$ gives $\mathbf{A}$'s inverse — multiply the adjoint by the reciprocal of the determinant
- A good way to check hand-calculated work is to verify $\mathbf{A} @ \mathbf{A}^{-1} = \mathbf{I}$

LA_05_03_01.ipynb · From Formula to Code

- a) Extract minor matrices by calling `numpy.delete()` to remove the target row and column
- b) Compute each minor matrix's determinant to get all 9 minors
- c) Adjust each minor's sign by $(-1)^{i+j}$ to get the 9 cofactors
- d) Use Laplace expansion along row 1 to compute $\det(A)$
- e) Assemble the 9 cofactors into matrix C, then transpose to get $\text{adj}(A)$
- f) Divide $\text{adj}(A)$ by $\det(A)$ to get the inverse, and compare against `numpy.linalg.inv()`

The Adjoint Method: Clear Formulas, Heavy Computation

$$\mathbf{A}^{-1} = \text{adj}(\mathbf{A}) / \det(\mathbf{A}) = \mathbf{C}^T / \det(\mathbf{A})$$

- The adjoint method breaks solving for the inverse into clear steps — minor matrices, minors, cofactors, the determinant, and the transpose to the adjoint — each with its own algebraic meaning
- But as the matrix grows, hand-computing the cofactor matrix quickly gets expensive; for efficiency, stability, and numerical precision, prefer ready-made tools like `numpy.linalg.inv()`
- Later in this book, we'll cover a second hand method — row reduction — which reaches the same destination by a different route

Key Terms in This Section

Adjoint Matrix $\text{adj}(A)$ — the transpose of the cofactor matrix, satisfying $A^{-1} = \text{adj}(A)/\det(A)$

Minor Matrix A_{ij} — what remains of A after deleting row i and column j

Cofactor C_{ij} — the sign-adjusted minor, $C_{ij} = (-1)^{i+j}M_{ij}$

Inverse of a 2×2 Matrix $A^{-1} = 1/(ad-bc) [d \ -b; -c \ a]$ — the adjoint method simplified for 2×2

Row Reduction Method A second hand method for the inverse, introduced later in this book

Cofactor Matrix C — built from every element's cofactor

Minor M_{ij} — the determinant of the minor matrix A_{ij}

Laplace Expansion Computing a determinant by expanding along any row or column using cofactors

Transpose Swapping a matrix's rows and columns; $\text{adj}(A) = C^T$ is one application

numpy.linalg.inv() NumPy's built-in inverse function — faster, more stable, and more precise than hand computation

Companion Code and Further Reading

Companion Code · LA_05_03_01.ipynb

A full implementation of minor matrix extraction, minors and cofactors, the determinant, the adjoint, and the inverse

Further Reading · `numpy.linalg.inv()`

For efficiency, stability, and numerical precision, prefer NumPy's linear algebra functions for computing inverses

Further Reading · Row Reduction Method

A second hand method for the inverse, covered systematically later in this book

Open-source Repository

All code and figures for this book: <https://github.com/Visualize-ML>

Q1 – Q8

Q1 Use this section's method to compute the inverse of the 2×2 matrix $A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$.

Q3 Study `time.time()` on your own, and compare the runtime of `numpy.linalg.inv()` against a custom inverse function.

Q5 Write a function `get_minor_matrix(A, i, j)` that uses `numpy.delete()` to remove row i and column j , returning the minor matrix $A_{i,j}$.

Q7 Write a function `adjoint(A)` that calls `cofactor_matrix(A)` and transposes it, returning the adjoint matrix $\text{adj}(A)$.

Q2 Use this section's method to hand-solve the inverse of the 3×3 matrix $A = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 0 \end{bmatrix}$, and verify with NumPy.

Q4 Study row reduction as a way to hand-solve a matrix inverse on your own.

Q6 Write a function `cofactor_matrix(A)` that calls `get_minor_matrix()` for every element of square matrix A , building the full cofactor matrix C .

Q8 Write a function `inverse_by_adjoint(A)` that combines `adjoint(A)` with `numpy.linalg.det(A)` to implement the full adjoint method, and compare against `numpy.linalg.inv(A)`.

Q9 – Q16

Q9 Hand-solve the inverse of the 4×4 identity matrix I using this section's method, and verify the result is still I .

Q11 Write a function `verify_inverse(A)` that computes $A @ \text{inverse_by_adjoint}(A)$, checking whether it equals the identity matrix (tolerance $1e-9$).

Q13 Write a function `random_invertible_3x3()` that generates a random 3×3 matrix, checks (and regenerates if needed) that its determinant is nonzero, then solves its inverse with the adjoint method.

Q15 Write a function `cofactor_sign(i, j)` that returns $(-1)^{(i+j)}$, and use it to verify the signs of this section's 9 cofactors.

Q10 Construct a 3×3 diagonal matrix $D = \text{diag}(2, 3, 4)$, hand-solve its inverse with the adjoint method, and compare against simply taking reciprocals.

Q12 For this section's 3×3 matrix A , compute $\det(A)$ by expanding along row 2 and row 3 separately, and verify both give 12.

Q14 Use SymPy to compute the adjoint $\text{adj}(A)$ of a 3×3 matrix with symbolic entries, and compare against numpy's numeric result.

Q16 Construct a 3×3 matrix with determinant 0, try computing its inverse via $\text{adjoint}(A)/\det(A)$, observe what happens, and explain why.

Q17 – Q24

Q17 Write a function `inverse_2x2(a, b, c, d)` that directly applies the 2×2 inverse formula and returns the inverse.

Q19 Write a function `matrix_of_cofactors_check(A)` that verifies whether the cofactor matrix C satisfies $A @ C.T = \det(A) \times I$.

Q21 Write a function `adjoint_of_transpose(A)` that checks whether $\text{adj}(A.T)$ equals $\text{adj}(A).T$.

Q23 Write a function `inverse_via_cramer(A, b)` that solves the linear system $Ax=b$ using the adjoint/Cramer's-rule idea (hint: $x = A^{-1}b$).

Q18 Use `time.time()` to compare the adjoint method against `numpy.linalg.inv()` on a random 5×5 matrix.

Q20 For this section's matrix A , hand-verify: does the determinant become 0 if one row is replaced by another row (a singular matrix)?

Q22 Construct a 2×2 matrix, and compare directly applying $A^{-1} = \text{adj}(A)/\det(A)$ against computing it step by step with the adjoint method.

Q24 Combining what you learned, write a short summary (code comments are fine) comparing the pros and cons of the adjoint method versus `numpy.linalg.inv()`.

10 Prompts to Explore Further (1/2)

1

Explain why the denominator $ad-bc$ in the 2×2 inverse formula is exactly the matrix's determinant.

2

Explain the relationship between the adjoint matrix $\text{adj}(A)$ and the cofactor matrix C — why is $\text{adj}(A) = C^T$ rather than $\text{adj}(A) = C$?

3

Explain the difference between the minor $M_{i,j}$ and the cofactor $C_{i,j}$ — why does the cofactor need the sign $(-1)^{(i+j)}$?

4

Explain why computing a determinant only needs one row or column expanded, while assembling the cofactor matrix needs all 9 elements.

5

Explain the full adjoint-method workflow for finding an inverse, and which formula in this section each step corresponds to.

10 Prompts to Explore Further (2/2)

6 Explain why $A @ A^{-1} = I$ can be used to check whether a hand-computed inverse is correct.

7 Explain why the adjoint method's computational cost grows so quickly as the matrix grows from 2×2 to 3×3 to 4×4 .

8 Explain why, for efficiency, stability, and numerical precision, real code should prefer `numpy.linalg.inv()` over hand-implementing the adjoint method.

9 Explain the connection between the cofactor matrix C and the determinant $\det(A)$ covered in the previous section.

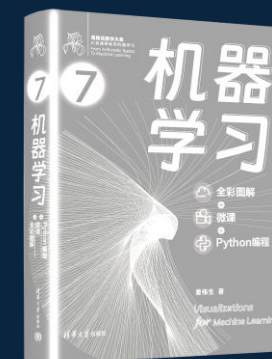
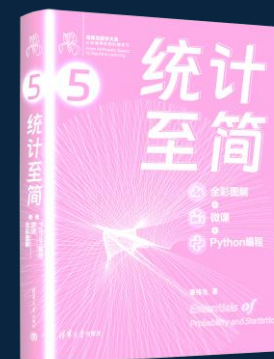
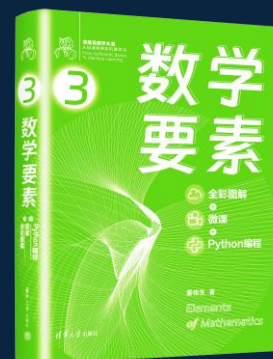
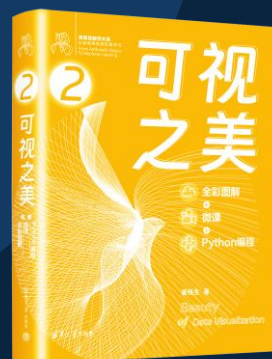
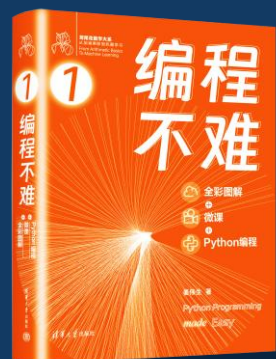
10 Combining what you learned, discuss the possible pros and cons of the adjoint method versus row reduction for hand-solving an inverse.

数学

数学基础

线性代数

概率统计



Python编程

数据可视化

工具

数据分析
图和网络

回归、降维
分类、聚类

实践

